It's important to note the behavior when the user doesn't have the claim. `FrequentFlyerHandler` doesn't do anything in this case; it returns a completed `Task` to satisfy the method signature.

NOTE Remember that if any of the handlers associated with a requirement passes, the requirement is a success. Only one of the handlers must succeed for the requirement to be satisfied.

This behavior, whereby you either call `context.Succeed()` or do nothing, is typical for authorization handlers. The following listing shows the implementation of `IsAirlineEmployeeHandler`, which uses a similar claim check to determine whether the requirement is satisfied.

Listing 24.9 `IsAirlineEmployeeHandler`

```
public class IsAirlineEmployeeHandler :
    AuthorizationHandler<AllowedInLoungeRequirement>                ❶
{
    protected override Task HandleRequirementAsync(                 ❷
        AuthorizationHandlerContext context,                        ❷
        AllowedInLoungeRequirement requirement)                     ❷
    {
        if(context.User.HasClaim(c => c.Type == "EmployeeNumber"))  ❸
        {
            context.Succeed(requirement);                           ❹
        }
        return Task.CompletedTask;                                  ❺
    }
}
```

❶ The handler implements AuthorizationHandler<T>.

❷ You must override the abstract HandleRequirementAsync method.

❸ Checks whether the user has the EmployeeNumber claim

❹ If the user has the necessary claim, marks the requirement as satisfied by calling Succeed

❺ If the requirement wasn't satisfied, does nothing